```
class DisjointSet
  #type HashMap[Element, Info]
  parentsMap
```

#1 The constructor for the `Info` class just takes an of the disjoint set.
#2 Validate the argument.
#3 Initially, an element is assigned to the singleton tree rooted at the element itself.
#4 The rank of the subtree is initially 1, because it only contains one element.

This `Info` class models (the info associated with) a node of the partitions' tree. It is, to all purposes, just a container for two values: the root of the tree and the rank (i.e. size) of the tree rooted at current element.

In the root property, we won't store references to other nodes. Instead, we will directly store the (index of) element itself, that we then use as a key to a `HashMap`, exactly as we have shown in the previous sections.

If we were actually modeling a tree data structure, this design would result in imperfect encapsulation. But we are rather just using the `Info` class as a *tuple* to gather all the properties associated with an element.

Most implementations for disjoint-set would use two arrays for this. Since our implementation does not restrict keys to integers and we are using hash maps, we could as well define two `Maps` for element's roots and ranks. In doing so,